



PROJECT DOCUMENTATION

Workshop Operations Platform

A single workspace for Oakland Motor Care Ltd – work orders, vehicles, parts, sales, scrap yard, debts and finance, running on Next.js and Supabase.

VERSION

September 2026

STACK

Next.js 16 · Supabase

DELIVERED AS

Web app & installable PWA



CONTENTS

What's inside

This document describes what the platform does, how it is built, and how to run, extend and maintain it safely.

01	Overview	Purpose, users and scope	3
02	Functional modules	Every area of the sidebar	4
03	Work order lifecycle	Statuses, transitions and job types	6
04	Roles & permissions	Admin, Garage Manager and RLS	7
05	Architecture & stack	Frontend, backend, hosting	8
06	Data model & conventions	Money, numbering, soft delete, audit	9
07	Bulk data import	CSV pipelines for historical records	10
08	Development workflow	Setup, checks, migrations, deployment	11
09	Project history	Milestones from August to September 2026	12

01 · OVERVIEW

One place to run the workshop

The platform replaces paper job cards, spares day books and scrap ledgers with a single, permission-controlled system that works on desktop and mobile.

6

SIDEBAR AREAS

30+

SCREENS

61

DB MIGRATIONS

53

PERMISSION KEYS

Who uses it

ADMINISTRATOR**Owners & management**

Full access: users and roles, settings, voids and reversals, admin purge, audit logs and all reporting.

GARAGE MANAGER**Day-to-day operators**

Opens work orders, records sales and scrap entries, receives stock and takes payments within the permissions granted.

What it covers

Workshop

Customers, vehicles, a gate vehicle register, work orders, services and technicians.

Stock & sales

Parts ledger, stock movements, low-stock alerts, suppliers, purchase orders and the spares sales day book.

Scrap yard

Daily purchase and sale records, stock position, reconciliation, finances and per-type rates.

Debts

A register of money owed from job cards and sales, with a payment pipeline and reports.

Finance

Quotations, invoices, payments (cash and M-Pesa) and multi-item general receipts.

System

Reports, notifications, audit logs, settings, users & roles, and install / update guidance.

Design principles

Database-enforced rules. Status transitions, numbering, stock changes and access control live in Postgres functions and Row-Level Security, so the rules hold no matter which screen, import or device writes the data. **Paper-faithful layouts** – the Parts table and Sales day book mirror the ledgers staff already know.

02 · FUNCTIONAL MODULES

Modules at a glance

The sidebar is grouped into six areas. Each screen is shown only when the signed-in user holds the matching permission.

Operations

WORKSHOP FLOOR

The dashboard shows the workshop pulse – metrics, active jobs, items needing attention and quick actions (new work order, new sale, receive stock). Creating a work order automatically checks the vehicle in to the Vehicle Register.

Dashboard

Customers

Vehicles

Vehicle Register

Work Orders

Services

Technicians

Inventory

PARTS & SPARES

The Parts table follows the paper ledger's columns, including remarks and date purchased. Every stock change is written as a traceable movement. Sales follow the Spares Sales Day Book format, support editing and voiding, require a technician for garage-workshop sales, and can be linked to a debt.

Sales

Parts

Stock Movements

Low Stock

Suppliers

Procurement

Scrap Yard

SIX SUB-MODULES

Daily purchase, sale, expense and cash records with bulk daily entry and editing. Stock position is tracked per scrap type, and the Reconciliation screen compares expected with counted stock, records a reason, and prints a clearance receipt. Ledger calculations start on 2 April 2026.

Overview

Daily Records

Stock Position

Reconciliation

Finances

Reports

Types & Rates

Debts

ACCOUNTABILITY

A business-wide Debt Register fed by job cards and sales. Payments recorded against a debt update its balance and notify staff when it is settled. All debt reporting is consolidated here.

Overview

Debt Register

Reports



Finance

BILLING & CASH

Quotations with approve / reject, invoices with voiding and overdue tracking, payments with reversal, and general receipts that can carry several services without being tied to a vehicle or work order. Payments accept cash and M-Pesa, with reference numbers and idempotency keys to prevent double-posting.

[Quotations](#)[Invoices](#)[Payments](#)[Receipts](#)

System

ADMINISTRATION

Reports across operations, finance, sales, inventory, procurement, technicians and scrap, with CSV, Excel and image exports. Notifications fire on real events – low stock, approvals, assignments, payments and new entries – with unread counts on the bell and sidebar. An audit log records sensitive actions.

[Reports](#)[Notifications](#)[Audit Logs](#)[Settings](#)[Users & Roles](#)[Install & updates](#)

Cross-cutting features

Global search

Grouped results across customers, vehicles, work orders, parts, sales, invoices, suppliers, quotations and purchase orders.

Installable PWA

Manifest, branded icons and an install banner that returns until the app is installed. A dedicated page explains installing and updating.

Printable documents

Branded purchase orders, suggested purchase orders, sales PDFs and reconciliation receipts.

Admin purge

Administrators can remove test or incorrect records, cascading safely through linked vehicles and entries.

Branded error pages

Custom error boundary, 404 page and logo loaders keep the experience consistent.

Responsive layout

A sticky sidebar on desktop and a collapsible menu on mobile, with paginated lists throughout.

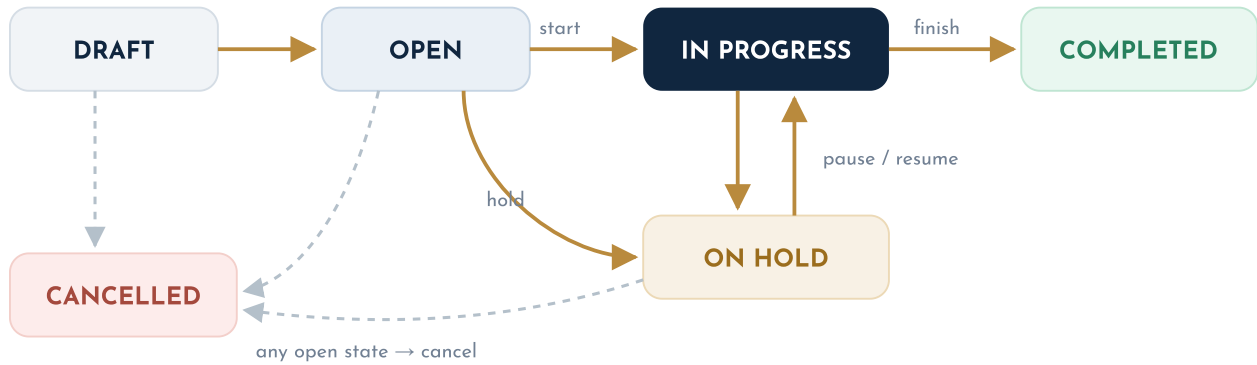
Payments & M-Pesa

M-Pesa transactions are stored with their reconciliation identifiers, and sales, job cards and payments record the payment method and M-Pesa reference. Daraja calls are designed to run server-side only (Supabase Edge Functions), so API keys never reach the browser.

03 · WORK ORDER LIFECYCLE

From check-in to completion

Work orders move through a fixed set of statuses. The allowed moves are enforced in the database by `transition_job_status()` and mirrored in `lib/constants.ts`.



FROM	ALLOWED NEXT STATUSES
Draft	Open, Cancelled
Open	In progress, On hold, Cancelled
In progress	Completed, On hold, Cancelled
On hold	In progress, Cancelled
Completed / Cancelled	Final – no further transitions

What a work order holds

Inspection

Condition per area – engine, transmission, brakes, suspension, electrical, tyres, body & paint.

Work items

Labour, parts (each linked to its own sale), deferred parts and other charges.

Quality check

Sign-off that the work is complete, parts are fitted correctly and the complaint is addressed.

Job types: Service, Repair, Diagnostics, Body work, Accident repair, Maintenance, Agricultural machinery, Tractor repair, Equipment repair, Other. Numbers run from **JB-001** and are assigned by the database.

04 · ROLES & PERMISSIONS

Access that the database enforces

There are two roles. Permissions are granted per role and checked twice: in the interface, to hide screens and buttons, and in Postgres through Row-Level Security and privileged functions.

ADMIN

Administrator

Holds the wildcard `*` and passes every permission check. The first account created during system initialisation becomes Administrator.

MANAGER

Garage Manager

Receives the set of permission keys assigned to the role. Administrators create accounts manually and can deactivate them; inactive accounts see an "account inactive" screen.

Permission groups

AREA	KEYS
Workshop	customer.*, vehicle.*, job.view / create / update / assign / complete / deliver
Vehicle Register	vehicle_register.view / record / manage
Inventory	inventory.view / create / update / issue / receive / adjust
Sales	sales.view / create / void / price.override / report
Procurement	supplier.*, purchase_order.view / create / approve / receive
Finance	quotation.*, invoice.view / create / update / void, payment.view / create / reverse
Scrap	scrap.view / record / manage
System	dashboard.view, report.view / export, audit.view, settings.manage, users.manage

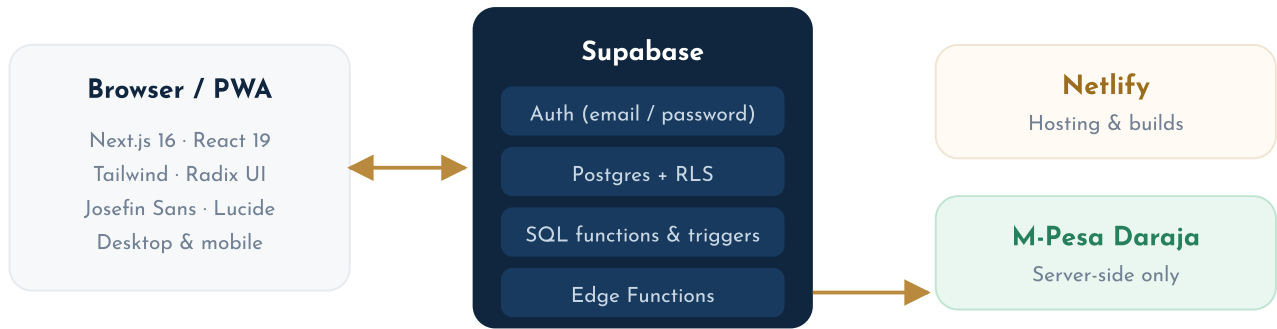
Security hardening

- 1 RLS on every business table.** Authenticated access is evaluated by a permission helper function inside Postgres.
- 2 Privileged operations run as functions** – status transitions, numbering, sale voids, payments and purges – so clients cannot bypass the rules.
- 3 Hardened system initialisation** guards against re-running first-time setup with a stale "initialized" flag.
- 4 Security review fixes** closed three access-control gaps; the integration suite covers RBAC, auth, payments and stock.

05 · ARCHITECTURE & STACK

How it's built

A client-rendered Next.js application talks directly to Supabase. Business rules live in the database, which keeps the frontend thin and every write consistent.



LAYER	TECHNOLOGY
Framework	Next.js 16 (App Router), React 19, TypeScript 5.9
Styling	Tailwind CSS 3, hand-tuned global CSS, Radix UI primitives, Lucide icons, Josefin Sans
Backend	Supabase – Auth, Postgres with RLS, SQL functions, triggers, Edge Functions
Data export	ExcelJS, CSV (UTF-8 with BOM), html-to-image, print-ready HTML
Testing	Vitest (unit and integration), Playwright (end-to-end)
Hosting	Netlify with <code>@netlify/plugin-nextjs</code>

Repository layout

PATH	CONTENTS
<code>app/</code>	Main application shell and screens, install page, error and 404 pages, manifest, icons, global CSS
<code>components/</code>	Feature modules – sales, scrap, debts, vehicle-register, parts, jobcards – plus shared <code>ui/</code>
<code>lib/</code>	Supabase client, permissions, constants, formatting, CSV import planners, PWA helpers, types
<code>supabase/migrations/</code>	Ordered SQL migrations: schema, RLS, functions and triggers
<code>tests/</code>	<code>unit/</code> , <code>integration/</code> (runs against Supabase) and <code>e2e/</code>

06 · DATA MODEL & CONVENTIONS

Rules the data follows

The schema is built from 61 ordered migrations. These conventions keep figures correct and history intact.

MONEY

Integer minor units

All amounts are stored as whole cents: KES 1,500.00 is stored as `150000`. This avoids floating-point rounding errors in totals and balances.

NUMBERING

Database-assigned numbers

Work orders (JB-001...) and sales receive sequential numbers from Postgres, so two devices can never take the same number.

HISTORY

Soft deletion

Customers, vehicles and jobs are flagged as deleted rather than erased, keeping invoices and reports consistent.

IDENTITY

Unique registrations

Vehicle registration numbers are uniquely indexed, case-insensitively, among active records.

STOCK

Traceable movements

Every receipt, issue, sale, void and adjustment writes a movement linked to its source record. Shelf and system counts are shown side by side.

PAYMENTS

Idempotent posting

Payments carry idempotency keys and M-Pesa references, so a retried request cannot post twice.

Core entities

DOMAIN	TABLES
People & access	profiles, roles, permissions, role_permissions, user_roles
Workshop	customers, vehicles, vehicle register, services, technicians, job cards with assignments, labour, parts, history and inspection records
Stock	parts, stock movements, suppliers, purchase orders, sales and sale items
Scrap	scrap types and rates, daily records, purchases, sales, expenses, reconciliations
Money	quotations, invoices, invoice items, payments, M-Pesa transactions, general receipts, debts
System	notifications, audit logs, settings

07 · BULK DATA IMPORT

Bringing paper records in

Historical ledgers are loaded through CSV uploads. Each pipeline validates every row before anything is written, then reports exactly which rows failed and why.

PARTS**Parts ledger**

Imports the stock register, including shelf counts, remarks and date purchased.

SALES**Spares Sales Day Book**

Imports historical sales in the day-book format, with an editable shelf count during review.

SCRAP**Scrap daily records**

Imports daily purchases, sales and cash against a validated schema, with row-specific error reasons.

WORK ORDERS**Historical job cards**

Imports past work orders with their customers and vehicles.

How an import runs

- 1 **Download the template** from the upload dialog so the columns match exactly.
- 2 **Upload the CSV.** It is parsed in the browser, and garbled encodings and real-world formatting quirks are normalised.
- 3 **Review the plan.** A pure planning function in `lib/*Import.ts` classifies each row as create, update or error. These planners are unit-tested against real exported files.
- 4 **Confirm.** Valid rows are written through the same database functions the screens use, so numbering, stock and audit rules still apply.

Ready-to-import files for parts, daily sales, scrap daily records and daily job cards sit at the project root and can be re-run safely against a fresh environment.

08 · DEVELOPMENT WORKFLOW

Running and shipping changes

Every change goes through the same checks before it is pushed. Netlify builds and deploys from `main`.

Scripts

COMMAND	PURPOSE
<code>npm run dev</code>	Start the local development server
<code>npm run typecheck</code>	TypeScript check with <code>tsc --noEmit</code>
<code>npm run lint</code>	ESLint (flat config, ignores nested build output)
<code>npm test</code>	Unit tests in <code>tests/unit</code>
<code>npm run test:integration</code>	Tests against a live Supabase project: auth, RBAC, numbering, transitions, payments, stock
<code>npm run test:e2e</code>	Playwright core-workflow test
<code>npm run build</code>	Production build

Release checklist

- 1 **Typecheck, lint and unit tests** – run in that order; all must pass cleanly.
- 2 **Build** – run with placeholder `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY` when real credentials are not available locally.
- 3 **Push to `main`** – Netlify builds with `npx next build` and publishes.

Database migrations

Migrations are applied by hand. New SQL files are added to `supabase/migrations/` with a timestamped name, then pasted into the Supabase Dashboard SQL editor and run there. Apply them in filename order, and keep `lib/constants.ts` in step with any status or transition rules they change.

Environment

Copy `.env.example` to `.env` and set the Supabase URL and anon key. **Service-role keys and M-Pesa credentials** belong only in Supabase Edge Function secrets, never in the browser bundle.

09 · PROJECT HISTORY

How the platform grew

Key milestones from the first commit in August 2026 to the current release.

Foundation

Core schema, Supabase auth, RLS and a Phase 1 test suite; rebrand to Oakland Motor Care Ltd with the real logo.

Job card workflow

Status pipeline, child tables, numbering, terms and other charges.

Sales, PWA & M-Pesa

Sales module, installable app experience, and hardened auth and M-Pesa handling.

Scrap & Vehicle Register

Scrap management and the gate vehicle register; RBAC simplified to Administrator and Garage Manager.

Traceability & audit

Traceable stock movements, a full-app audit fixing 22 issues, and per-part job-card sales.

Scrap expansion

Split into six sub-modules; stock reconciliation with printable receipts; multi-item general receipts.

Notifications & security

Real-event notifications with unread counts; security-review fixes; hardened initialisation.

Debts

Job-card debt pipeline and the business-wide Debt Register with reports.

Paper-faithful ledgers

Parts and Sales reworked to match the paper books; bulk uploads with downloadable templates; admin purge.

Next.js 16 & Josefin Sans

Framework upgrade to Next.js 16 and React 19; Josefin Sans adopted across the app; list pagination.

Scrap refinements

Daily-record editing, Excel export, admin cash controls; the ledger now starts on 2 April 2026.

Current release

On-hold status, bulk work-order upload, automatic vehicle check-in, an Install & updates page and branded logo loaders.



Workshop Operations Platform



Built for Oakland Motor Care Ltd. Created and powered by Qeem Labs Ltd.